Programiranje za web
Predavanje 1: Arhitektura backenda i kostur projekta

doc. dr. sc. Tomislav Petković mag. ing. Josip Torić

08.10.2026.





Sveučilište u Zadru
Universitas Studiorum
Jadertina | 1396 | 2002 |

Plan predavanja

1. Od zahtjeva do odgovora: što *backend* radi i kroz što jedan zahtjev prolazi
2. Ugovor prema klijentu: rute, statusni kodovi, opis API-ja
3. Kostur projekta: struktura, konfiguracija, pokretanje
4. Dijagnostika i kvaliteta: zapisnik, pogreške, statička analiza
5. Rad s verzijama koda

Na kraju predavanja projekt je pokrenut: baza u kontejneru i backend koji odgovara na zahtjev.

Predmet kolegija

Web aplikacija je program kojemu korisnik pristupa preglednikom, a čiji podatci ostaju pohranjeni i nakon što korisnik zatvori karticu. Za to su nam potrebni:

- **frontend** (klijentski dio): prikaz podataka u pregledniku i unos korisnika
- **backend** (poslužiteljski dio): provjera identiteta i prava, poslovna pravila, pristup podatcima
- **baza podataka**: trajna pohrana i ograničenja nad podatcima

Kolegij obrađuje sva tri dijela i sučelje kojim međusobno komuniciraju.

Ishodi učenja

Nakon ovog predavanja student može:

- objasniti kroz koje provjere prolazi zahtjev koji mijenja podatke
- opisati ugovor prema klijentu: metodu, putanju i statusni kod
- postaviti razvojnu okolinu: baza u kontejneru i konfiguracija iz varijabli okoline
- objasniti kako se aplikacija sastavlja i pokreće

Od zahtjeva do odgovora

Troslojna arhitektura i podjela odgovornosti, HTTP i što putuje mrežom, jednostavan backend, koraci obrade zahtjeva.

Troslojna arhitektura



Preglednik ne pristupa bazi izravno. Svaki podatak koji korisnik vidi prošao je kroz *backend*, koji je pritom provjerio tko je korisnik i što smije.

Podjela odgovornosti

- Baza trajno čuva podatke, neovisno o tome radi li aplikacija u tom trenutku. Ona je jedino mjesto na kojem podatci doista postoje.
- Backend je mjesto na kojem se pravilo provodi: prije svake izmjene provjerava se tko šalje zahtjev, smije li to i dopušta li trenutno stanje podataka.
- *Frontend* pravila prikazuje, ali ih ne provodi. Onemogućen gumb korisniku štedi uzaludan pokušaj, ali ne sprječava zahtjev poslan mimo sučelja.

HTTP

HTTP (engl. *HyperText Transfer Protocol*) je dogovor o tome kakve poruke dva programa izmjenjuju preko mreže i kojim redom. Nije ni jezik ni *library*, nego pravila kojih se oba sugovornika drže, pa se sporazumijevaju bez obzira na to u čemu su napisani.

Tri svojstva iz kojih slijedi sve ostalo:

- **Razgovor uvijek počinje klijent.** Poslužitelj sam od sebe ne šalje ništa, nego čeka zahtjev i na njega odgovara.
- **Jedan zahtjev, jedan odgovor.** Nema odgovora bez zahtjeva ni dvaju odgovora na isti zahtjev.
- **Poslužitelj ne pamti prethodne zahtjeve.** Svaki stiže sam za sebe, pa sve što je za odluku potrebno mora biti u njemu.

HTTP zahtjev i odgovor

Zahtjev je tekst: metoda, putanja, zaglavlja i, po potrebi, tijelo.

```
GET /clubs/12/lifters HTTP/1.1
Host: api.primjer.hr
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Odgovor je također tekst: statusni kod, zaglavlja i tijelo.

```
HTTP/1.1 200 OK
Content-Type: application/json

[{"id": 31, "ime": "Ana Horvat"}]
```

Frontend šalje prvi tekst, *backend* ga prima i sastavlja drugi.

Primjer jednostavnog backenda

```python
from fastapi import APIRouter

router = APIRouter()

@router.get("/competitions")
def list_competitions():
   return [
      {"id": 7, "naziv": "Zimski kup", "datum": "2027-03-14"},
      {"id": 8, "naziv": "Ljetni kup", "datum": "2027-06-06"},
   ]
```

Framework (ovdje FastAPI) povezuje par metode i putanje s funkcijom. Funkcija vraća podatke, a framework ih pretvara u JSON, dodaje zaglavlja i statusni kod `200`. Funkcija pridružena putanji naziva se **ruta**.

Obrada zahtjeva

Popis natjecatelja jednoga kluba nije javan, a prijava natjecatelja k tome mijenja podatke. Za obradu takvog zahtjeva i sastavljanje odgovora moramo:

1. provjeriti oblik i tipove poslanih podataka
2. utvrditi tko šalje zahtjev
3. provjeriti smije li taj korisnik do tih podataka
4. provjeriti dopuštaju li poslovna pravila izmjenu u ovom trenutku
5. upisati promjenu tako da pri pogrešci ne ostane polovičan trag

Svaki zahtjev koji mijenja podatke prolazi kroz ove korake.

Ruta za provjeru stanja

```
# app/main.py
app.include_router(health_router, prefix="/health", tags=["health"])
```

```
# app/routers/health.py
@router.get("/")
def health():
   return {"status": "ok"}
```

Odgovor na `GET /health` potvrđuje da:

- proces radi
- usmjeravanje zahtjeva radi
- sastavljanje aplikacije nije prijavilo pogrešku

Ugovor prema klijentu

Metoda i putanja, statusni kodovi i njihovi razredi, opis API-ja.

Metoda i putanja

Putanja imenuje ono nad čime se radi, a metoda kaže što se radi. Par metode i putanje jednoznačno određuje rutu.

Metoda	Putanja	Što poslužitelj radi
`GET`	`/clubs/12/lifters`	vraća popis natjecatelja kluba 12
`POST`	`/clubs/12/lifters`	dodaje novog natjecatelja u klub 12
`GET`	`/clubs/12/lifters/31`	vraća podatke natjecatelja 31
`PATCH`	`/clubs/12/lifters/31`	mijenja podatke natjecatelja 31
`DELETE`	`/clubs/12/lifters/31`	briše natjecatelja 31

Ista putanja s različitim metodama vodi na različite funkcije. To je svojstvo HTTP-a, a ne izbor aplikacije.

Oblikovanje putanja

HTTP ne propisuje kako putanja treba izgledati. Aplikacija smije rute nazvati i `/getLifters` i `/l?id=31`. Ustaljenog se oblika držimo zato što se iz njega vidi što ruta radi i bez dokumentacije.

Konvencija

- Putanja sadrži imenice u množini (`/lifters`), a ne glagole (`/getLifters`). Glagol pripada metodi (`GET`, `POST`), a ne putanji.
- Pojedinačni zapis dobiva se dodavanjem oznake (`/lifters/31`).
- Pripadnost se izražava ugnježđivanjem (`/clubs/12/lifters`).

Aplikacija koja odstupa od toga radi jednako dobro. Cijena odstupanja plaća se u vremenu svakoga tko popis ruta prvi put čita.

Razredi statusnih kodova

Svaki odgovor počinje trozanamenkastim statusnim kodom. Standard HTTP-a (RFC 9110) dijeli kodove u pet razreda prema prvoj znamenki:

Razred	Značenje
1xx	obavijest o tijeku obrade, odgovor još slijedi
2xx	zahtjev je primljen, razumljiv i obrađen
3xx	potrebna je daljnja radnja, najčešće odlazak na drugu adresu
4xx	zahtjev je pogrešan ili se ne može ispuniti takav kakav je
5xx	zahtjev je bio ispravan, ali ga poslužitelj nije uspio obraditi

Standard traži da klijent razumije razred svakoga koda, i onda kad sam kod ne poznaje. Kod iz razreda `4xx` koji klijent ne prepoznaje smije obraditi kao `400`.

Primjeri statusnih kodova

Kod	Značenje	Kod	Značenje
200	obrađeno, tijelo slijedi	405	metoda nad tom putanjom nije podržana
201	nastao je novi zapis	409	sukob sa stanjem resursa
204	obrađeno, tijela nema	415	oblik sadržaja nije podržan
400	zahtjev je neispravan	422	sadržaj se ne može obraditi
401	prijava nedostaje ili ne vrijedi	500	nepredviđeno stanje u poslužitelju
403	zahtjev je odbijen	503	poslužitelj trenutno ne poslužuje
404	na toj putanji nema ničega		

Opis API-ja nastaje iz koda

OpenAPI je specifikacija: strojno čitljiv opis API-ja u JSON-u, s popisom ruta, metoda, oblika tijela zahtjeva i odgovora te statusnih kodova. Swagger UI je samo jedan prikazivač tog opisa, ne isto što i OpenAPI.

Taj opis ne piše se ručno. FastAPI ga sastavlja iz tipova i deklaracija koje su već napisane, pa ne može zastarjeti neovisno o kodu.

- `/openapi.json` je sam opis
- `/docs` je Swagger UI nad njim
- `/redoc` je drugi prikazivač istog opisa

Dostupnost opisa u produkciji

```
app = FastAPI(
   title=config.APP_NAME,
   docs_url="/docs" if config.ENV == "dev" else None,
   redoc_url=None,
)
```

Konvencija

Dokumentacija se u produkciji često gasi, da popis ruta i oblik tijela zahtjeva ne budu javno dostupni. Nije obavezno i mnogi projekti je ostavljaju otvorenom; opis tada služi i onima koji se na API spajaju izvana.

Kostur projekta

Struktura repozitorija i backenda, konfiguracija, sastavljanje aplikacije, baza u kontejneru, pokretanje.

Struktura repozitorija

```
repo/
  api/ # backend (FastAPI)
  web/ # frontend (Vue)
  docker-compose.yml # database service
  .env.example # configuration template
  .gitignore
  README.md
```

Izbor, ne pravilo

Backend i frontend ovdje stoje u istom repozitoriju. Promjena koja dira obje strane tada je jedan *commit* umjesto dvaju usklađenih u dvama repozitorijima. Odvojeni repozitoriji imaju svoje razloge, primjerice različite timove i različit ritam izdanja.

Struktura backenda

```
api/
  app/
    main.py # creates the application
    core/
      config.py # configuration from the environment
      errors.py # domain errors and handlers
      logging.py # logging setup
    routers/ # health.py
    models/
    schemas/
    services/
    repositories/
  requirements.txt
```

Konfiguracija iz varijabli okoline

```python
# app/core/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    APP_NAME: str = "Powerlifting Registrations API"
    ENV: str = "dev"
    DATABASE_URL: str
    JWT_SECRET: str = "change-me"

settings = Settings()
```

- Vrijednosti se čitaju na jednome mjestu, pa `os.getenv` ne treba pozivati po kodu.
- `.env` ostaje na računalu i ulazi u `.gitignore`; u repozitorij ide `.env.example` s nazivima varijabli i bezopasnim vrijednostima.
- Prijelaz s razvojne okoline na produkcijsku mijenja vrijednosti varijabli, a ne kod.

Sastavljanje aplikacije

Aplikacija nastaje tako da se frameworku prijave rute, postavke i zajedničke ovisnosti.

```python
# app/main.py
from fastapi import FastAPI
from app.core.config import Settings, settings
from app.routers.health import router as health_router

def create_app(config: Settings = settings) -> FastAPI:
    app = FastAPI(title=config.APP_NAME, debug=config.ENV == "dev")
    app.include_router(health_router, prefix="/health", tags=["health"])
    return app

app = create_app()
```

Posljednji redak stvara primjerak koji poslužitelj pokreće. Isti se poziv može ponoviti drugdje i s drugim postavkama, primjerice `create_app(test_settings)` u testovima.

Baza u kontejneru

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: pl_reg
      POSTGRES_USER: pl_user
      POSTGRES_PASSWORD: pl_pass
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Zapis određuje istu inačicu PostgreSQL-a za sve, neovisno o operacijskom sustavu. Imenovani volumen `pgdata` čuva podatke između pokretanja; brisanjem volumena baza se vraća u prazno stanje.

Pokretanje i provjera

```
docker compose up -d db # database

cd api
uv venv
source .venv/bin/activate
uv pip install -r requirements.txt
uvicorn app.main:app --reload
```

Alat `uv` objedinjuje stvaranje virtualne okoline i instalaciju paketa; `requirements.txt` ostaje nepromijenjen.

Provjera: `http://127.0.0.1:8000/health` vraća `{"status": "ok"}`, a `/docs` prikazuje popis ruta.

Dijagnostika i kvaliteta

Zapisnik rada, dosljedan oblik pogreške, statička analiza i formatiranje.

Što ulazi u zapisnik

Aplikacija usput zapisuje što je radila. Taj zapisnik (engl. *log*) jedini je trag kojim se poslije rekonstruira što se dogodilo.

Zapisuje se	**Ne zapisuje se**
ruta, metoda i statusni kod odgovora	lozinke i tokeni, ni u dijelovima
ispis iznimke s popisom poziva, u razvojnoj okolini	tijelo zahtjeva s osobnim podatcima
domenski događaji, primjerice odjava natjecatelja	podatci po kojima se osoba može identificirati

Postavljanje zapisivanja

```python
# app/core/logging.py
import logging

def setup_logging(debug: bool = False) -> None:
   logging.basicConfig(
      level=logging.DEBUG if debug else logging.INFO,
      format="%(asctime)s %(levelname)s %(name)s: %(message)s",
   )

# in a service module:
logger = logging.getLogger(__name__)
logger.info("club created: id=%s", club.id)
```

`logger` se dohvaća preko `__name__`, pa svaki redak nosi naziv modula iz kojega dolazi. Zapis ide na standardni izlaz, odakle ga preuzima okolina u kojoj proces radi.

Dosljedan oblik pogreške

Svaki neuspjeh vraća tijelo istoga oblika:

```
{
  "code": "registration_closed",
  "message": "Rok za prijave je istekao."
}
```

- `code` je stalan i namijenjen kodu koji odlučuje što učiniti.
- `message` je namijenjen čovjeku i smije se mijenjati.

Iznimka koju aplikacija podiže nosi oba polja i statusni kod.

Konvencija

Uz ta dva polja često dolazi i treće, s popisom polja koja nisu prošla provjeru. Potrebno je kad sučelje poruku prikazuje uz svaki unos u obrascu, a ne samo jednu na vrhu.

Što se vidi bez pokretanja koda

- **Promjena koja to nije.** Dva autora imaju različito postavljene uređivače: jedan uvlači s četiri razmaka, drugi s dva i pri spremanju preslaguje uvoze. Kad drugi otvori i spremi tuđu datoteku, `git diff` pokazuje četrdeset izmijenjenih redaka, a ponašanje je isto. Stvarne promjene treba tražiti među njima.
- **Pogreška vidljiva iz samoga koda.** Neiskorišten uvoz, varijabla kojoj je vrijednost dodijeljena a nikad pročitana, grana do koje izvođenje ne može doći, krivo napisan naziv.

Prvo uklanja alat koji kod zapisuje u jedan stil, drugo pronalazi alat koji ga pregledava. Ruff radi oboje: `ruff format .` i `ruff check .`.

Oba se pokreću lokalno i pri svakoj promjeni u repozitoriju. Ono što alat može provjeriti ne treba provjeravati čovjek.

Rad s verzijama koda

Commit kao zapis jedne promjene, oblik i opseg poruke, čitanje razlike, naredbe za pregled povijesti.

Commit kao zapis jedne promjene

Povijest repozitorija je zapis o tome kako je aplikacija nastala. Koliko je taj zapis upotrebljiv, ovisi o tome koliko je promjena stavljeno u jedan commit.

- Commit koji dira jednu temu može se pročitati u cijelosti i, ako zatreba, poništiti sam za sebe.
- Commit s petnaest datoteka i tri nepovezane promjene ne može ni jedno ni drugo.
- Poruka opisuje namjeru (`add config loading from environment`), a ne postupak (`izmjene`).

Kad se pojavi pogreška koje prije nije bilo, pretraga povijesti pokazuje u kojem se commitu ponašanje promijenilo. Taj je odgovor koristan samo ako commit sadrži jednu promjenu.

Oblik poruke commita

Poruke u ovom projektu imaju isti oblik: vrsta promjene, dio sustava u zagradi, pa namjera.

Vrsta	Kad se koristi
`feat`	nova mogućnost vidljiva prema van
`fix`	ispravak ponašanja koje je bilo pogrešno
`refactor`	promjena koda bez promjene ponašanja
`test`	dodani ili izmijenjeni testovi
`docs`	dokumentacija
`chore`	alati, konfiguracija, zavisnosti

Opseg promjene u poruci

U zagradi stoji dio sustava, `api`, `web` ili `ci`, a za promjene u korijenu repozitorija zagrada izostaje. Iz same se poruke time vidi koju stranu promjena dira:

```
feat(api): add JWT utilities (create/decode access + refresh tokens)
```

Konvencija: Conventional Commits

Ovaj oblik nije dio gita niti ga ijedan standard propisuje. Riječ je o raširenom dogovoru koji olakšava pretragu povijesti i samostalno sastavljanje popisa promjena. Projekt koji odabere drukčiji oblik ništa ne krši, sve dok ga se drži dosljedno.

Kako čitati razliku

1. Poruka commita: koji je cilj promjene.
2. Popis datoteka: koje su nove, a koje izmijenjene.
3. Promjene vidljive prema van: nova ruta, promijenjen oblik tijela zahtjeva, novo polje u odgovoru.
4. Promjene u konfiguraciji: varijable okoline, opis usluga, zavisnosti.
5. Tek na kraju pojedinosti izvedbe.

Redoslijed je namjeran: prva tri koraka kažu mijenja li se ugovor prema klijentu, a to je jedino što odmah utječe na frontend.

Korisne naredbe

```
# history, one line per commit
git log --oneline --decorate

# what a single commit changed
git show <commit>

# difference between two commits
git diff <commit1>..<commit2>

# uncommitted changes in the working tree
git diff
```

Zadnja naredba pokazuje što je promijenjeno, a još nije zabilježeno. Pregled te razlike prije svakog commita hvata zaboravljene ispise, privremene izmjene i datoteke koje ne bi trebale ući u repozitorij.

Sažetak

- Backend prima HTTP zahtjev i sastavlja odgovor. U najmanjem obliku to je funkcija pridružena putanji.
- Čim podatci nisu javni i čim se mijenjaju, uz to dolaze utvrđivanje pošiljatelja, provjera prava, provjera oblika poslanih podataka, poslovna pravila i transakcija.
- Statusni kod nosi značenje: `401` je neutvrđen identitet, `403` nedostatak ovlasti, `409` sukob sa stanjem resursa.
- Konfiguracija dolazi iz varijabli okoline, pa prijelaz u produkciju ne mijenja kod; `.env` ostaje izvan repozitorija.
- Aplikacija zapisuje što je radila, a pogreška prema klijentu ima uvijek isti oblik. Ono što alat provjeri bez pokretanja koda ne provjerava čovjek.
- Commit je zapis jedne promjene: poruka kaže zašto, a razlika pokazuje što.